



Apache Spark

Лекция 4





Я Александр
Бадьин

Ведущий разработчик
Inference-платформы

11:25



План занятия

Устройство DataFrame

Схема данных

UDF

Группировки

MLlib



Устройство DataFrame

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - **Распределен**

`df.rdd.getNumPartitions()` – получить число партиций

`pyspark.sql.functions.spark_partition_id()` – колонка с id партиции

`df.repartition(1000)` – перераспределить данные

`df.repartition(10, ["title"])` – перераспределить данные по значению колонки

`df.coalesce(10)` – уменьшает число партиций по возможности без shuffle

Малое число партиций - плохо

Демо

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - Распределен
 - **Иммутабелен**

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - Распределен
 - **Иммутабелен**

С DataFrame ассоциирован план того, как получить его данные

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - Распределен
 - **Иммутабелен**

С DataFrame ассоциирован план того, как получить его данные
`df.explain()`

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - Распределен
 - **Иммутабелен**

С DataFrame ассоциирован план того, как получить его данные
`df.explain()`

При добавлении трансформаций план обновляется

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - Распределен
 - **Иммутабелен**

С DataFrame ассоциирован план того, как получить его данные
`df.explain()`

При добавлении трансформаций план обновляется

Логический план преобразуется в физический через оптимизатор Catalyst

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - Распределен
 - **Иммутабелен**

С DataFrame ассоциирован план того, как получить его данные
`df.explain()`

При добавлении трансформаций план обновляется

Логический план преобразуется в физический через оптимизатор Catalyst

При потере executor spark знает, как пересчитать данные заново

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - Распределен
 - Иммутабелен
- **Обладает схемой данных**

Устройство DataFrame

- Устроен поверх RDD (Resilient Distributed Dataset)
 - Распределен
 - Иммутабелен
- **Обладает схемой данных**

`df.printSchema()` – вывести схему данных

`df.schema` – получить схему в виде `org.apache.spark.sql.types.StructType`

Типы данных Spark

Все типы находятся в модуле `pyspark.sql.types`

Типы данных Spark

Все типы находятся в модуле `pyspark.sql.types`

Простые типы:

- `BooleanType()` - bool
- `ByteType()`, `ShortType()`, `IntegerType()`, `LongType()` – int
- `FloatType()`, `DoubleType()` – float
- `StringType()`, `VarcharType(max)`, `CharType(n)` – str
- `BinaryType()` – bytes
- `TimestampType()`, `TimestampNTZType()`, `DateType()`, `DayTimeIntervalType()` – datetime.*

Типы данных Spark

Все типы находятся в модуле `pyspark.sql.types`

Составные типы:

- `ArrayType(elementType)` – list, tuple или array
- `MapType(keyType, valueType)` – dict
- `StructType(fields)` – list или tuple
 - `StructField(name, dataType)` – тип для описания полей структуры

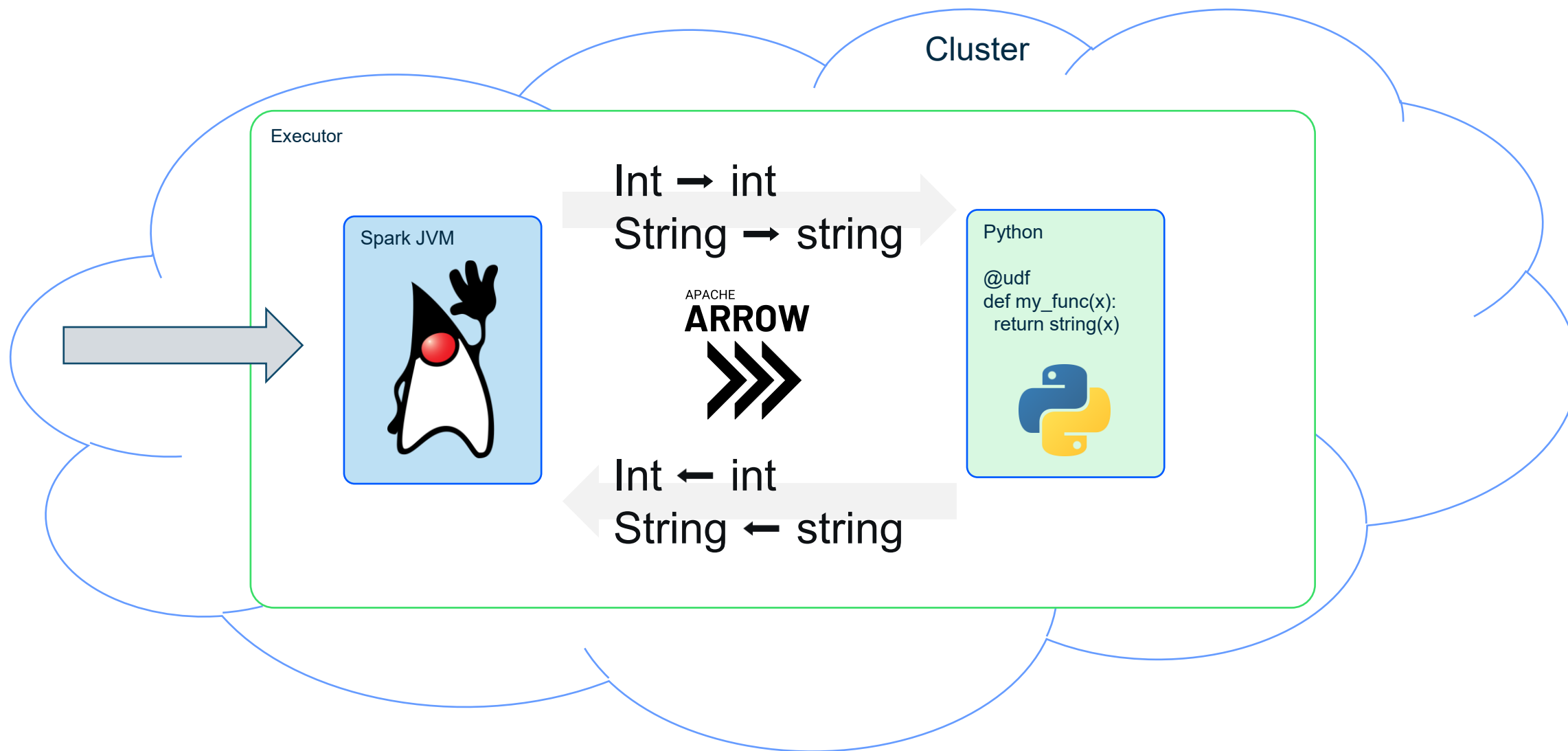
Демо

User Defined Functions

- Позволяют применять свой код для преобразования DataFrame
- Используется функция F.udf
- `def udf(f=None, returnType=T.StringType())`
- `@udf(returnType=StringType())`
`def normalize(x):`
 `return unicodedata.normalize("NFC", x)`

Демо

Проблемы использования UDF



Ключевые моменты UDF

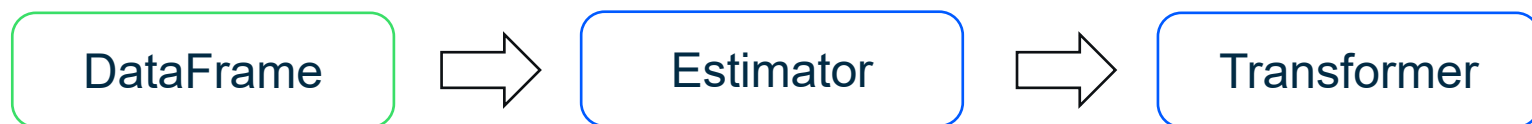
- UDF не позволяет использовать оптимизации Spark
- Требуется передача данных между процессом executor и python
- Runtime Python потребляет память
- Зависимости (окружение) нужно дотаскивать до executor
- Не использовать UDF без крайней необходимости

Группировки

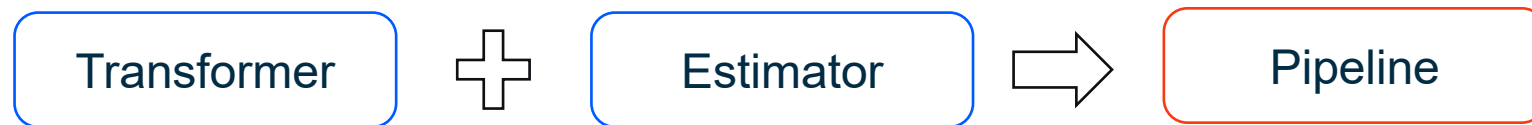
- Необходимы, когда нужно уменьшить число данных
- Оставляет **одну** строку для каждого уникального набора значений колонок
- Для всех колонок (sum, avg, count, min, max)
- Выбор по колонкам `.groupby(...).agg(...)`

Демо

MLlib



MLlib



Выводы

- Spark составляет план исполнения действий, получая возможность воспроизводить данные
- Важной частью DataFrame API было введение схемы данных
- Spark позволяет исполнять кастомные функции
- UDF имеет свои минусы
- Spark предоставляет API для запуска простых ML алгоритмов